System Design Document: UnifiedDocumentViewer

Document type: System Architecture and Design
System name: UnifiedDocumentViewer
Architecture style: UI–API separation with a service-based backend

1. Introduction

1.1 Purpose

UnifiedDocumentViewer is a document-viewing application consisting of two separate projects:

- **UnifiedDocumentViewer:** The user interface responsible for displaying documents and handling user interactions.
- **UnifiedDocumentViewerAPI:** The backend API responsible for providing document data and application services.

The system includes mock JSON data for sales and service documents, allowing document-viewing functionality to be developed and tested before integration with a production data source.

1.2 Objectives

- Provide a unified interface for viewing different types of documents.
- Separate presentation logic from backend business logic.
- Support sales and service document data.
- Enable independent development and testing of the UI and API.
- Provide an architecture that can be extended to support additional document types and data sources.

2. System Architecture

2.1 High-Level Architecture

```
                  | HTTP / HTTPS API requests
                  v
+-----------------------------------+
|       UnifiedDocumentViewerAPI    |
|                                   |
|  Controllers                      |
|  Models                           |
|  Services                         |
|  MockData                          |
|   - sales-documents.json           |
|   - service-documents.json         |
|  Program.cs                       |
+-----------------+-----------------+
                  |
                  v
+-----------------------------------+
|          Document Data            |
|                                   |
|  Sales Documents                  |
|  Service Documents                |
+-----------------------------------+
```

The UI communicates with the API through HTTP requests. The API processes requests and retrieves document data through its service layer. In the current design, the document data is supplied by JSON files in the `MockData` directory.

This architecture keeps the user interface independent of the underlying document data implementation.

3. Project Structure

3.1 UnifiedDocumentViewer — UI Application

Component	Responsibility
Controllers	Handle UI routes and coordinate requests between views and services.
Models	Define data structures used by the UI.
Pages	Contain page-level UI components and user interaction logic.
Services	Encapsulate API communication and client-side application operations.
Views	Define the presentation templates rendered to users.
`wwwroot`	Stores static assets such as CSS, JavaScript, images, and libraries.
`Program.cs`	Configures application startup, dependency injection, middleware, and routing.

The UI application is responsible for rendering document lists, presenting document details, handling user interactions, and displaying loading states or errors.

3.2 UnifiedDocumentViewerAPI — Backend Application

Component	Responsibility
Controllers	Expose HTTP endpoints for document operations.

Component	Responsibility
Models	Define API request, response, and document data structures.
Services	Implement document retrieval, processing, and business rules.
`MockData`	Contains JSON files used as sample document data.
`Program.cs`	Configures API startup, dependency injection, middleware, and endpoint mapping.

The API provides a centralized interface for document retrieval and processing. Its controllers should delegate business logic to services rather than directly implementing all data-processing logic.

3.3 Mock Data

The API currently contains two data files:

- `sales-documents.json`: Sample data for sales-related documents.
- `service-documents.json`: Sample data for service-related documents.

The service layer should load and process this data, while controllers expose the appropriate results through API endpoints.

The JSON files act as a mock data source and can later be replaced with a database, external document repository, or another approved data provider without requiring major changes to the UI.

4. Core System Components

4.1 User Interface Layer

The UI provides the user-facing document-viewing experience.

Responsibilities include:

- Displaying available documents.
- Allowing users to select document categories or individual documents.
- Requesting document data from the API.
- Presenting document details in a consistent layout.
- Handling API errors and empty results.

4.2 API Controller Layer

The controller layer exposes endpoints for document-related operations.

Potential endpoints include:

Method	Endpoint	Purpose
GET	`/api/documents`	Retrieve documents across supported categories, if implemented.
GET	`/api/documents/sales`	Retrieve sales documents.
GET	`/api/documents/service`	Retrieve service documents.

Method	Endpoint	Purpose
GET	`/api/documents/{id}`	Retrieve a specific document, if supported.

These endpoints are proposed design examples. Actual routes should match the existing controller implementation.

4.3 Service Layer

The service layer encapsulates document retrieval and business logic.

Responsibilities include:

- Reading data from the configured data source.
- Mapping stored records into API models.
- Applying supported filtering and validation rules.
- Returning consistent results to controllers.
- Providing a clear integration point for future data sources.

4.4 Model Layer

Models define the structure of document information shared between application components. A document model may include fields such as:

- Document ID
- Document type
- Document title or number
- Document date
- Document status
- Additional type-specific metadata

These fields are illustrative; the final model must reflect the actual contents of the JSON files and the application's requirements.

5. Main Data Flow

5.1 Document Retrieval Flow

1. The user opens the UnifiedDocumentViewer UI.
2. The UI requests a document list or a specific document.
3. The UI service sends an HTTP request to the backend API.
4. The API controller validates and processes the request.
5. The controller calls the appropriate document service.
6. The service retrieves the relevant information from the JSON mock data.
7. The API returns a structured response.
8. The UI processes the response and displays the documents.

```
User
  |
  v
UI Page / View
```

```
  |
  v
UI Service
  |
  v
API Controller
  |
  v
Document Service
  |
  v
JSON Mock Data
  |
  v
API Response
  |
  v
UI Document Display
```

6. Technology and Integration

The project structure suggests an application organized around .NET-style entry points and separated UI and API projects. The exact framework versions and UI technology should be confirmed from the project files.

The architecture should support:

- HTTP communication between the UI and API.
- Dependency injection for service registration.
- Configuration-based API URLs.
- JSON serialization and deserialization.
- Environment-specific configuration for development and production.

If the UI and API are hosted on different origins, configure CORS on the API for the specific UI origins that are permitted.

7. Security Considerations

Before exposing the application publicly, consider the following:

- **Authentication:** Require authentication if documents are not intended for unrestricted public viewing.
- **Authorization:** Ensure users can access only documents they are permitted to view.
- **Transport security:** Use HTTPS in deployed environments.
- **Secrets management:** Keep credentials, API keys, and connection strings out of source control.
- **Input validation:** Validate document IDs, filters, and other request parameters.
- **Error handling:** Avoid returning stack traces or sensitive implementation details to clients.
- **Data protection:** Check the mock JSON files for confidential, personal, or business-sensitive information before publishing the repository.
- **CORS:** Restrict cross-origin access to trusted origins when required.

Making the Git repository public is separate from making the running application publicly accessible. Public repository access exposes source code; public deployment exposes the application through a reachable URL.

8. Deployment Architecture

A typical deployment can use two independently hosted applications:

```
Public / Authorized Users
          |
          v
      HTTPS URL
          |
          v
 UnifiedDocumentViewer
          |
          | HTTPS API calls
          v
 UnifiedDocumentViewerAPI
          |
          v
 Configured Document Data Source
```

Deployment requirements include:

- Publish the UI and API using their respective project configurations.
- Configure the UI with the correct API base URL.
- Configure the API's allowed origins and environment-specific settings.
- Verify that the API can access its JSON data files after deployment.
- Enable HTTPS and appropriate authentication and authorization.
- Test document retrieval, error handling, and access controls in the deployed environment.

The specific hosting platform and deployment pipeline remain to be selected.

9. Error Handling and Reliability

The application should handle common failure scenarios consistently:

- API unavailable or unreachable.
- Invalid document identifiers.
- Missing or malformed JSON files.
- Empty document collections.
- Unsupported document categories.
- Unexpected errors during document retrieval.

The API should return suitable HTTP status codes and safe error responses. The UI should present understandable messages and avoid displaying raw server exceptions.

10. Future Enhancements

Potential future improvements include:

- Replacing mock JSON data with a persistent database or document management system.
- Adding search, sorting, filtering, and pagination.
- Introducing document previews and downloads where required.
- Supporting additional document categories.
- Implementing centralized logging and monitoring.

- Adding automated unit and integration tests.
- Establishing a CI/CD pipeline for automated builds and deployments.
- Adding caching where appropriate for frequently accessed documents.

11. Assumptions and Open Questions

This document is based on the supplied folder structure rather than a review of the complete source code.

The following items need confirmation:

- Which .NET version and UI framework are used?
- What API endpoints and document models already exist?
- Are the JSON files the only data source, or is another backend planned?
- Should the application be publicly accessible to everyone, or restricted to authenticated users?
- Where will the UI and API be deployed?

12. Conclusion

UnifiedDocumentViewer uses a separated UI and API architecture to provide a maintainable foundation for document viewing. The UI handles presentation and user interaction, while the API manages document retrieval and business logic. JSON mock data currently supports development and testing.

The separation of responsibilities makes it possible to evolve the data source, add document categories, and deploy the UI and API independently as the project grows.

Recommended Next Step

Add this document to the root of your Git repository as `SYSTEM_DESIGN.md`. Before treating it as the final architecture, compare the proposed data flow and endpoints with your actual source code.

For an implementation-accurate system design, review the `.csproj` files, API controllers, and service implementations to confirm the actual endpoints, models, dependencies, and deployment configuration.